

Tier 2: Dynamic Landmark Supermodel

Abdul Ahad. Student ID: G00486649

10 July 2026

Table of Contents

0.1	Overview	1
0.2	Load Libraries	1
0.3	Locate Data and Set Output Directories	1
0.4	Load Landmark Stack	2
0.5	Feature Design	2
0.6	Landmark Time and Interactions	2
0.7	Patient-Grouped 5-Fold CV	2
0.8	Separate Per-Landmark Comparator	3
0.9	AUC by Landmark	3
0.10	AUC by Landmark Figure	4
0.11	Done	4

0.1 Overview

This script fits a dynamic-prediction landmark supermodel (van Houwelingen 2007). A single penalized-logistic model is fit across the full stack of landmarks at {6, 12, 18, 24, 36, 48} hours, with the landmark time s and feature-by- s interactions as covariates. Evaluating it at the current hour yields an hourly-updating sepsis risk, a genuine real-time model built entirely from classical statistics. Leakage-safe evaluation uses patient-grouped CV folds and only past-information features.

0.2 Load Libraries

```
suppressPackageStartupMessages({ library(glmnet); library(pROC) })
set.seed(486649)
```

0.3 Locate Data and Set Output Directories

```
find_data <- function(f) {
  cands <- c(file.path("../..//processed_data", f),
             file.path("processed_data", f), f)
  hit <- cands[file.exists(cands)][1]
  if (is.na(hit)) stop("cannot find ", f); hit
}
OUT <- dirname(find_data("11_landmark_stack.parquet"))
FIG <- if (dir.exists("figure")) "figure" else { dir.create("figure"); "figure" }
```

0.4 Load Landmark Stack

```
st <- as.data.frame(arrow::read_parquet(find_data("11_landmark_stack.parquet")))
cat(sprintf("Loaded stack: %d rows, %d stays, landmarks {%s}\n",
           nrow(st), length(unique(st$stay_id)),
           paste(sort(unique(st$landmark_time)), collapse = ",")))
```

```
## Loaded stack: 207010 rows, 48829 stays, landmarks {6,12,18,24,36,48}
```

0.5 Feature Design

Vitals and scores receive median imputation, consistent with the Tier 1 scheme. Labs and slopes are median/zero-imputed and paired with binary “measured” indicators to preserve the informative-missingness signal.

```
vitals <- c("hr","resp_rate","map","spo2","temp_c","gcs")
labs   <- c("lactate","creatinine","wbc","pf_ratio")
slopes <- c("hr_slope6","resp_rate_slope6","map_slope6","spo2_slope6")
scores <- c("sofa_total","qsofa","sirs")

df <- data.frame(row.names = seq_len(nrow(st)))
for (v in c(vitals, scores)) { x <- st[[v]]; x[is.na(x)] <- median(x, na.rm=TRUE); df[[v]] <- x }
for (v in labs) { x <- st[[v]]; df[[paste0(v,"_msg")]] <- as.integer(is.na(x));
                  x[is.na(x)] <- median(x, na.rm=TRUE); df[[v]] <- x }
for (v in slopes) { x <- st[[v]]; df[[paste0(v,"_msg")]] <- as.integer(is.na(x));
                    x[is.na(x)] <- 0; df[[v]] <- x }
df$age <- st$age; df$female <- as.integer(st$gender == "F")
base_feats <- names(df)
```

0.6 Landmark Time and Interactions

The model includes standardized landmark time s , its quadratic term, and all base-feature-by- s interactions. Standardizing s places the interaction terms on a comparable scale to the main effects under the elastic-net penalty.

```
s <- st$landmark_time
s_z <- as.numeric(scale(s))
df$s <- s_z
df$s2 <- s_z^2
for (v in base_feats) df[[paste0(v, "_xS")]] <- df[[v]] * s_z
X <- as.matrix(df)
y <- st$onset_within_h
cat(sprintf("Supermodel design: %d features (incl. s, s^2, %d s-interactions)\n",
           ncol(X), length(base_feats)))
```

```
## Supermodel design: 56 features (incl. s, s^2, 27 s-interactions)
```

0.7 Patient-Grouped 5-Fold CV

Each stay recurs across multiple landmarks, so folds are assigned at the patient level to prevent information leakage.

```

ids <- unique(st$stay_id)
idf <- setNames(sample(rep(1:5, length.out = length(ids))), ids)
fold <- idf[as.character(st$stay_id)]

oos <- numeric(nrow(X))
for (k in 1:5) {
  tr <- fold != k
  cv <- cv.glmnet(X[tr,], y[tr], family="binomial", alpha=0.5,
                  nfolds=5, standardize=TRUE)
  oos[!tr] <- as.numeric(predict(cv, X[!tr,], s="lambda.min", type="response"))
}

```

0.8 Separate Per-Landmark Comparator

A separate penalized model fit independently at each landmark (no pooling, no s terms) serves as a comparator to quantify the discrimination gain from the supermodel's pooled structure.

```

Xb <- as.matrix(df[, base_feats])
oos_sep <- numeric(nrow(X))
for (sl in sort(unique(s))) {
  idx <- which(s == sl)
  fk <- fold[idx]
  for (k in 1:5) {
    tr <- fk != k
    if (sum(y[idx][tr]) < 10) { oos_sep[idx][!tr] <- mean(y[idx][tr]); next }
    cv <- cv.glmnet(Xb[idx,][tr,], y[idx][tr], family="binomial",
                    alpha=0.5, nfolds=5, standardize=TRUE)
    oos_sep[idx][!tr] <- as.numeric(predict(cv, Xb[idx,][!tr,],
                                           s="lambda.min", type="response"))
  }
}

```

0.9 AUC by Landmark

Out-of-sample AUROC at each prediction hour for both models, saved to `tier2_auc_by_landmark.csv` and `tier2_supermodel_oos.csv`.

```

auc_at <- function(p, sl) {
  i <- s == sl
  as.numeric(auc(roc(y[i], p[i], quiet=TRUE, direction="<")))
}
grid <- sort(unique(s))
tab <- data.frame(
  landmark_h = grid,
  n_at_risk = as.integer(table(s)[as.character(grid)]),
  n_onset = as.integer(tapply(y, s, sum)[as.character(grid)]),
  auc_supermodel = sapply(grid, function(sl) auc_at(oos, sl)),
  auc_separate = sapply(grid, function(sl) auc_at(oos_sep, sl)))
tab$auc_supermodel <- round(tab$auc_supermodel, 4)

```

```

tab$auc_separate <- round(tab$auc_separate, 4)
overall <- as.numeric(auc(roc(y, oos, quiet=TRUE, direction="<")))
cat(sprintf("\nOverall supermodel out-of-sample AUROC (pooled): %.4f\n", overall))

##
## Overall supermodel out-of-sample AUROC (pooled): 0.7973

print(tab, row.names = FALSE)

## landmark_h n_at_risk n_onset auc_supermodel auc_separate
##          6    48829    3330         0.7712         0.7753
##         12    44576    1369         0.7340         0.7337
##         18    39990     773         0.6957         0.7044
##         24    33610    353         0.6445         0.6579
##         36    23238    232         0.7109         0.7138
##         48    16767    152         0.7537         0.7277

write.csv(tab, file.path(OUT, "12_tier2_auc_by_landmark.csv"), row.names=FALSE)
arrow::write_parquet(data.frame(stay_id=st$stay_id, landmark_time=st$landmark_time,
                                onset_within_h=y, risk=oos),
                    file.path(OUT, "12_tier2_supermodel_oos.parquet"))
write.csv(data.frame(metric = "overall_pooled_auroc", value = overall),
          file.path(OUT, "12_overall_auroc.csv"), row.names = FALSE)

```

0.10 AUC by Landmark Figure

```

png(file.path(FIG, "12_auc_by_landmark.png"), width=1100, height=850, res=200)
plot(tab$landmark_h, tab$auc_supermodel, type="b", pch=19, col="steelblue",
     ylim=range(c(tab$auc_supermodel, tab$auc_separate, 0.5)),
     xlab="Prediction hour s (landmark)", ylab="Out-of-sample AUROC (12h horizon)",
     main="Dynamic AUC(s): landmark supermodel")
lines(tab$landmark_h, tab$auc_separate, type="b", pch=1, lty=2, col="grey40")
abline(h=0.5, lty=3, col="grey70")
legend("topright", c("Supermodel (pooled + s)", "Separate per-landmark"),
     col=c("steelblue", "grey40"), pch=c(19,1), lty=c(1,2), bty="n", cex=0.8)
dev.off()

```

```

## pdf
## 2

```

0.11 Done

```

cat("\nTier 2 complete.\n")

```

```

##
## Tier 2 complete.

```